

Neural Network Basics

How Artificial Neurons Learn

■ NEURAL NETWORKS

■ LEARNING OUTCOME

By the end of this module you will understand the perceptron, activation functions, backpropagation, and build multilayer networks with Keras to solve classification and regression tasks.

■ Skills You Will Gain

- Understand perceptron and activation functions intuitively
- Build multilayer networks with Keras Sequential API
- Apply ReLU, sigmoid, and softmax activation functions
- Use backpropagation and Adam optimiser
- Prevent overfitting with dropout and batch normalisation
- Monitor training with callbacks: EarlyStopping, ReduceLROnPlateau

■ Neural networks are the backbone of modern AI — from ChatGPT to image recognition to medical diagnosis. This module builds the mental model that makes everything else click.

SECTION 1

From Neuron to Network

1	Single Neuron output = activation($w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + \text{bias}$)
2	Activation Non-linear function that decides whether a neuron 'fires'. Without it, any deep network collapses to one linear layer.
3	Layers Input layer -> Hidden layers (feature extraction) -> Output layer (prediction)
4	Forward Pass Data flows forward: input x weights + bias -> activation -> next layer -> loss
5	Loss Function Measures prediction error. MSE for regression, Cross-entropy for classification.
6	Backpropagation Computes gradient of loss w.r.t. every weight using chain rule — flows backward
7	Weight Update Optimiser uses gradients to adjust weights. Adam: $w = w - \text{lr} \cdot \text{gradient}$

SECTION 2

Activation Functions

	Formula	Use Case
	$\max(0, x)$	Hidden layers — fast, no vanishing gradient, default
	$\max(0.01x, x)$	When ReLU neurons 'die' (always output 0)
	$1/(1+e^{-x})$	Binary output layer — probability [0, 1]
	$e^{x_i} / \sum(e^{x_j})$	Multi-class output — probabilities summing to 1
	$(e^x - e^{-x}) / (e^x + e^{-x})$	RNNs and hidden layers — zero-centred
	approx $x \cdot \text{sigma}(1.702x)$	Transformers (BERT, GPT) — smooth ReLU variant

SECTION 3

Building with Keras

```
import tensorflow as tf from tensorflow.keras import layers, models model = models.Sequential([ #
Input + first hidden layer layers.Dense(256, activation='relu', input_shape=(n_features,)),
layers.BatchNormalization(), # normalise layer inputs -> faster training layers.Dropout(0.3), #
randomly zero 30% of neurons -> prevent overfit layers.Dense(128, activation='relu'),
layers.Dropout(0.2), layers.Dense(64, activation='relu'), # Output layer layers.Dense(n_classes,
activation='softmax') # 'sigmoid' for binary ]) model.compile( optimizer='adam',
```

```
loss='sparse_categorical_crossentropy', # 'binary_crossentropy' for binary metrics=['accuracy'] )
model.summary() # print architecture with parameter counts
history = model.fit( X_train, y_train,
epochs=100, batch_size=32, validation_split=0.2, callbacks=[
tf.keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True),
tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5) ] )
```

SECTION 4

Regularisation Techniques

	What It Does	When to Use
	Randomly zeros p% of neurons each training step — forces redundant learning	Dense layers; p=0.2-0.5
n	Normalises layer activations to mean~0, std~1 — stabilises training	After Dense/Conv layers
	Adds penalty for large weights: loss += lambda*sum(w^2)	When overfitting, many params
	Stops training when validation loss stops improving	Always — saves compute
n	Artificially expands training set with transformations	Image, text tasks
teau	Halves learning rate when validation metric stalls	When training plateaus

■ PRACTICE Q & A

- Q1. What would happen if you removed all activation functions from a deep neural network?

A: Every layer would just be a linear transformation, and stacking them collapses to a single linear function. No matter the depth, the model could only learn linear relationships.
- Q2. What does dropout do and why does it prevent overfitting?

A: Randomly disables a fraction of neurons during each training step, forcing the network to learn redundant representations that don't rely on specific neurons. Acts like an ensemble of smaller networks.
- Q3. What is vanishing gradient and how does ReLU solve it?

A: In deep networks, sigmoid/tanh gradients approach zero, making early layers learn extremely slowly. ReLU gradient is constant (1 for positive inputs), allowing gradients to flow back without shrinking.
- Q4. What does batch normalisation do and why is it useful?

A: Normalises layer inputs to zero mean and unit variance during training. Speeds up training, allows higher learning rates, reduces sensitivity to initialisation, and acts as mild regularisation.
- Q5. What does EarlyStopping(patience=10, restore_best_weights=True) do?

A: Monitors validation loss. If it doesn't improve for 10 epochs, training stops and the model reverts to the best weights seen during training — preventing overfitting and saving compute.

■ Neural Network Cheat Sheet	
<code>models.Sequential([])</code>	Stack layers in order
<code>layers.Dense(n, activation)</code>	Fully connected layer
<code>layers.Dropout(0.3)</code>	30% dropout rate
<code>layers.BatchNormalization()</code>	Normalise layer inputs
<code>activation='relu'</code>	Hidden layer default
<code>activation='softmax'</code>	Multi-class output
<code>activation='sigmoid'</code>	Binary output
<code>optimizer='adam'</code>	Adaptive learning rate
<code>loss='sparse_categorical_crossentropy'</code>	Multi-class loss

<code>loss='binary_crossentropy'</code>	Binary classification loss
<code>EarlyStopping(patience=10)</code>	Stop when val loss stalls
<code>ReduceLROnPlateau(factor=0.5)</code>	Halve LR when plateau